目录

第一章　引言 **5**

第二章　**Linux** 的哲学与发行版 **7**

第三章　**Shell** 与命令行基础 **9**

第四章　**Linux** 文件系统 **17**

第五章　重定向、管道与错误处理 **29**

第六章　**Shell** 脚本与参数 **35**

第七章　流程控制与条件测试 **47**

第八章　算术扩展、函数与数组 **57**

第一章　引言

本书后续章节大致按照「系统背景 → Shell 入门 → 文件系统 → 权限与链接 → 重定向与管道 → 脚本与控制结构 → 高级特性」的顺序展开，便于课堂教学与自学。

silde 1

Introduction au système Linux

- Système d’exploitation multi-utilisateurs (connexion simultanée de plusieurs utilisateurs) et multi-tâches (décomposition d’une application en plusieurs tâches qui s’exécutent simultanément).
- L’environnement standard de programmation est le C et plus récemment le C++.
- L’environnement réseau est construit sur les protocoles TCP/IP.

Linux 系统导论

- 这是一款多用户与多任务操作系统。
- 多用户特性：支持多个用户同时连接登录系统；多任务特性：可将单个应用程序拆解为多个任务，这些任务能够同时并发执行。
- 标准编程环境为 C 语言，近年来 C++ 也纳入其核心标准编程体系；其网络环境基于 TCP/IP 协议栈构建。

第二章　Linux 的哲学与发行版

在本章中，我们从宏观视角理解 Linux：它为何被设计成「一切皆文件」的世界，又为何会出现众多发行版。阅读本章时，可以思考三个问题：

- Linux 与传统的单机操作系统（如早期的 MS-DOS）相比，设计目标有何不同？
- 「一切皆文件」这一抽象思想，如何帮助我们统一地看待设备、进程和配置？
- 面对 Ubuntu、Fedora、openSUSE 等不同发行版，应该从哪些维度（目标用户、更新策略、稳定性等）来做选择？

Remark 从教学实践来看，很多同学在学习 Linux 时，会把「发行版」等同于「操作系统本身」。实际上，大多数发行版共享同一个 Linux 内核，只是在软件选择、默认配置和图形界面上有所差异。在阅读接下来的几页 `silde` 时，可以有意识地区分：*哪些内容是 Linux 内核的共性，哪些是发行版的个性。*

silde 2

Philosophie de Linux

- Un ensemble de règles assurent le fonctionnement et le succès de Linux. Le fameux paradigme « tout est fichier ». Toutes les fonctionnalités du système d'exploitation sont accessibles via de simples fichiers : périphériques, processus, mémoire, etc.
- Fini le développement seul dans son coin, la moindre petite ébauche de votre projet doit être soumise à la communauté afin d'avertir un maximum de personnes pouvant intervenir de différente manière au niveau du développement, soit en soulevant les bugs que vous vous attacherez à corriger le plus rapidement possible.

Linux 设计理念

- 一系列准则保障了 Linux 的运行与成功。其中最著名的范式便是「**一切皆文件**」。操作系统的所有功能均可通过普通文件进行访问：外设、进程、内存等。
- 摒弃闭门造车式的独立开发，你的项目哪怕只是最微小的雏形，也必须提交至社区，让尽可能多的相关人员能够以不同方式参与开发工作，例如指出程序漏洞，而你需尽快着手修复这些问题。

silde 3

La création de Linux

- En 1991, un étudiant finlandais nommé Linus Torvalds décide de créer un nouveau noyau basé sur Minix (un dérivé d' UNIX) afin de contourner ses limitations et celles de MS-DOS. Ce nouveau noyau est distribué gratuitement sur Internet en août 1991 dès sa version 0.1 sous le nom de «Linux».
- Actuellement, le noyau de Linux est à la révision 6.12.71 (12/02/2026) et est librement téléchargeable.

Linux 的诞生

- 1991 年，芬兰大学生林纳斯 托瓦兹（Linus Torvalds）决定基于 Minix（一种 UNIX 衍生系统）开发一款全新内核，以突破 Minix 与 MS-DOS 自身的局限。这款新内核于 1991 年 8 月以 0.1 版本在互联网上免费发布，并定名为「Linux」。
- 目前，Linux 内核版本为 6.12.71（2026 年 2 月 12 日），可自由下载使用。

silde 4

Les systèmes Linux du marché

- Ubuntu C'est une distribution commerciale orientée vers le grand public distribué gratuitement par Canonical, qui édite des versions stables tous les six mois (maintenues 9 mois).
- Fedora est une distribution communautaire supervisée par Red Hat, basée sur le système de gestion de paquetages logiciels RPM C'est une distribution qui met l'accent sur la nouveauté, ce qui signifie que ses logiciels sont très fréquemment mis à jour.
- openSUSE est une distribution communautaire destinée tant à un usage grand public qu'à un usage professionnel. C'est une distribution indépendante réputée pour ses outils de configurations et sa stabilité.

市面上的 Linux 系统

- Ubuntu：它是由 Canonical 公司免费发行、面向普通大众的商业发行版，该公司每六个月推出一个稳定版本（版本维护周期为 9 个月）。
- Fedora：它是由红帽（Red Hat）公司监管的社区发行版，基于 RPM 软件包管理系统。该发行版主打新技术、新特性，这意味着其内置软件会得到非常频繁的更新。
- openSUSE：它是同时面向普通用户与专业用途的社区发行版，作为一款独立发行版，它以配置工具完善和系统运行稳定而闻名。

silde 5

Linux sur votre ordinateur

- Installer Linux sur une partition de votre disque. Double boot Windows-Linux
- Utiliser une clé usb bootable Linux.
- Virtualiser Linux sur votre PC avec Virtuabox par exemple. Possibilité d' avoir Linux, windows et Mac OS en même temps.
- WSL (Windows Subsystem Linux) : Sous-système Windows pour Linux
- Sous MacOs:Zsh disponible directement.

在计算机中使用 Linux

- 在硬盘的一个分区上安装 Linux，实现 Windows 与 Linux 双系统启动。
- 使用 Linux 可启动 U 盘运行系统。
- 在电脑上通过 VirtualBox 等工具对 Linux 进行虚拟化，可同时运行 Linux、Windows 与 Mac OS 系统。
- WSL（Windows Subsystem for Linux）：Windows Linux 子系统。
- 在 macOS 系统下：可直接使用 Zsh。

第三章　Shell 与命令行基础

有了对 Linux 系统背景和主要发行版的认识，本章开始聚焦于「如何与 Linux 交互」。在多数教学和实际运维场景中，命令行 Shell 比图形界面更高效、更可组合。本章将围绕以下主线展开：

- 认识 Shell 的角色：既是命令解释器，也是脚本语言；
- 掌握基本命令（`ls`、`pwd`、`cal`、`date` 等）以及 `man`、`whatis`、`apropos` 等文档工具；
- 理解命令行的基本语法：命令、选项、参数的组合方式，以及提示符 `PS1` 的含义。

Remark 建议在学习本章内容时，同步打开一个真实的终端环境（Linux、WSL 或远程服务器均可）。每出现一个新命令，都亲自在终端输入并观察输出差异，例如对比 `ls` 与 `ls -l`，感受选项对输出格式的影响。只有「边学边敲」，Shell 的语法和习惯用法才会真正内化为自己的技能。

silde 6

Le Shell

- Sous Linux, on appelle Shell l’interpréteur de commandes qui fait office d’interface entre l’utilisateur et le système d’exploitation. Les shells sont des interpréteurs : cela signifie que chaque commande saisie par l’utilisateur (ou lue à partir d’un fichier) est syntaxiquement vérifiée puis exécutée.
- Il existe plusieurs interpréteurs Shell sous Linux : Bourne again shell (bash), Cshell, Korn shell, Z shell etc …
- Enconnaître un, permet d’accéder aisément aux autres.
- Tous les Shells peuvent cohabiter à l’intérieur d’un même système Linux. L’administrateur fixe le Shell initial de chaque utilisateur dans le fichier de définition des utilisateurs : /etc/passwd
- Un shell possède un environnement de travail et un langage de programmation.

命令行终端（Shell）

- 在 Linux 系统中，Shell 是指命令解释器，它充当用户与操作系统之间的交互接口。Shell 属于解释器：即用户输入的每一条命令（或从文件中读取的命令），都会先经过语法检查，随后被执行。
- Linux 系统中存在多种 Shell 解释器：Bourne again shell（bash）、C shell、Korn shell、Z shell 等。
- 掌握其中一种，即可轻松上手其他各类 Shell。
- 所有 Shell 均可在同一 Linux 系统中共存。管理员会在用户配置文件 /etc/passwd 中，为每个用户设定默认初始 Shell。
- 每个 Shell 都拥有独立的工作环境与编程语言。

silde 7

Unix Shell / Windows PowerShell 对比

PowerShell（命令本体）	PowerShell（别名）	Unix Shell 命令	中文功能描述
Get-ChildItem	gci, dir, ls	ls, dir	列出目录（当前目录）下的文件 / 目录
Get-Content	gc, type, cat	cat	获取文件内容
Get-Command	gcm	help, which	列出命令 / 查看命令信息
Get-Help	help, man	man	帮助文档
Clear-Host	cls, clear	clear	清屏
Copy-Item	cpi, copy, cp	cp	复制一个或多个文件 / 整个目录树
Move-Item	mi, move, mv	mv	移动文件 / 目录
Remove-Item	ri, del, erase, rmdir, rd, rm	rm, rmdir	删除文件 / 目录
Rename-Item	rni, ren, mv	mv	重命名文件 / 目录
Get-Location	gl, pwd	pwd	显示当前工作目录

silde 8

L' environnement de travail du Shell

- Le shell permet d' exécuter une commande en mode interactif ou bien par l'intermédiaire de fichiers de commandes (scripts).
- En mode interactif, le bash affiche à l' écran une chaîne d' appel (appelée également **prompt** ou **invite**), qui se termine par défaut par le caractère **#** suivi d' un caractère espace pour l' administrateur système (utilisateur **root**) et par le caractère **$** suivi d' un caractère espace pour les autres utilisateurs. Cette chaîne d' appel peut être relativement longue.
- Ex :

```
retraint@marylin:/home/retraint$
```

Shell 的工作环境

- Shell 支持以交互模式执行命令，也可通过命令文件（脚本）执行命令。
- 在交互模式下，**bash** 会在屏幕上显示一个命令提示符字符串（也称为 **prompt/invite**）：对于系统管理员（**root** 用户），该提示符默认以 **#** 字符加一个空格结尾；对于其他普通用户，默认以 **$** 字符加一个空格结尾。这个提示符字符串可能会相对较长。
- 示例：

```
retraint@marylin:/home/retraint$
```

silde 9

L' environnement de travail du Shell

- Le prompt est constituée du nom de connexion de l' utilisateur (retraint), du nom de la machine sur laquelle l' utilisateur travaille (marylin) et du chemin absolu du répertoire courant de l' utilisateur (/home/retraint). Il indique que le shell attend que l' utilisateur saisisse une commande et la valide en appuyant sur la touche entrée. Bash exécute alors la commande puis réaffiche la chaîne d' appel.
- Si l' on souhaite raccourcir la chaîne d' appel, il suffit de modifier la valeur de la variable prédéfinie du shell **PS1** (Prompt Shell 1).

- Ex :

```
retraint@marylin:/home/retraint$ PS1='$ '
```

Shell 的工作环境

- 命令提示符（**prompt**）由三部分构成：用户登录名（示例中的 **retraint**）、用户操作的主机名（示例中的 **marylin**）、以及用户当前目录的绝对路径（示例中的 **/home/retraint**）。该提示符表明 Shell 正等待用户输入命令，用户按下回车键即可确认执行命令。**bash** 会先执行该命令，随后重新显示命令提示符字符串。
- 若希望缩短命令提示符字符串，只需修改 Shell 的预定义变量 **PS1**（Prompt Shell 1，即一级提示符变量）的值即可。
- 示例：

```
retraint@marylin:/home/retraint$ PS1='$ '
```

bash 代码示例（可直接执行）：

```bash
# 示例：修改 PS1 变量以缩短 bash 命令提示符
retraint@marylin:/home/retraint$ PS1='$ '

# 执行上述命令后，命令提示符会立即从长格式变为极简格式：
# 原提示符：retraint@marylin:/home/retraint$
# 修改后提示符：$
```

silde 10

L’ environnement de travail du Shell

- Le Shell **bash** permet :
- le rappel des commandes précédentes (gestion de l’ historique) ; cela évite de taper plusieurs fois la même commande
- la modification en ligne du texte de la commande courante (ajout, retrait, remplacement de caractères)
- la gestion des travaux lancés en arrière-plan (appelés **jobs**) ; ceux-ci peuvent être démarrés, stoppés ou repris suivant les besoins
- l’initialisation adéquate de variables de configuration (chaîne d’appel de l’interpréteur, chemins de recherche par défaut) ou la création de raccourcis de commandes (commande **alias**).

Shell 的工作环境

- **bash** Shell 具备以下核心功能：
- 调用历史命令（历史记录管理）：避免重复输入相同的命令；
- 对当前输入的命令文本进行行内编辑（添加、删除、替换字符）；
- 管理后台运行的任务（称为「作业」**(jobs)**）：可根据需求启动、停止或恢复这些作业；
- 合理初始化配置变量（包括解释器的命令提示符字符串、默认搜索路径），或创建命令别名（快捷命令，使用 **alias** 命令）。

silde 11

Avantages du Shell

- C' est un langage interprété : les erreurs peuvent être facilement localisées et traitées. Des modifications de fonctionnalités sont facilement apportées à l' application sans qu' il soit nécessaire de recompiler et faire l' édition de liens de l' ensemble.
- Le shell manipule essentiellement des chaînes de caractères: Ceci a pour avantage d' éviter des erreurs de typage.
- Le langage est adapté au prototypage rapide d' applications : les tubes, les substitutions de commandes et de variables favorisent la construction d' une application par assemblage de commandes préexistantes dans l' environnement Unix.
- Le shell permet de connecter des composants écrits dans des langages différents.

Shell 的优势

- 它是解释型语言：错误可被轻松定位和处理。无需对整个程序重新编译、进行链接编辑，即可便捷地为应用修改功能。
- Shell 主要处理字符串：这一特性的优势在于能避免类型错误。
- 该语言适配应用的快速原型开发：管道（`tubes`）、命令替换与变量替换机制，支持通过组合 Unix 环境中已有的命令来构建应用程序。
- Shell 能够连接以不同编程语言编写的组件。

silde 12

Inconvénients du Shell

- Système d' exploitation écrit à l' origine par des développeurs pour des développeurs, les shells utilisent une syntaxe d' accès difficile pour le débutant.
- L' oubli ou l' ajout d' un caractère espace provoque facilement une erreur de syntaxe.
- Le bash possède plusieurs syntaxes pour implanter la même fonctionnalité.
- Certains caractères spéciaux, comme les parenthèses, ont des significations différentes suivant le contexte : les parenthèses peuvent introduire une liste de commandes, une définition de fonction ou bien imposer un ordre d' évaluation d' une expression arithmétique.

Shell 的缺点

- 该操作系统最初由开发者为开发者编写，Shell 所使用的语法对初学者而言上手难度较高。
- 遗漏或多添加一个空格字符，都极易引发语法错误。
- `bash` 实现同一功能可使用多种语法。
- 部分特殊字符（如括号）在不同上下文下含义不同：括号可用于表示命令列表、函数定义，或规定算术表达式的运算优先级。

silde 13

Syntaxe d’une ligne de commande

- Syntaxe des commandes
  - **nom_commande [options] [arguments]**
  - C’est une suite de mots séparés par un ou plusieurs blancs. On appelle blanc un caractère tab (tabulation horizontale) ou un caractère espace.
  - Un mot est une suite de caractères non blancs.
  - Une option est généralement introduite par le caractère tiret (ex : **-a**) ou dans la syntaxe par deux caractères tiret consécutifs (ex : **--version**).
  - Les arguments désignent les objets sur lesquels doit s’exécuter la commande.

命令行语法

- 命令语法格式：
  - 命令名 [选项] [参数]
- 命令行由一个或多个空白字符分隔的若干「单词」组成。这里的「空白字符（**blanc**）」指水平制表符（**tab**）或空格字符。
- 「单词（**mot**）」是一串连续的非空白字符。
- 选项（**option**）通常以单个连字符开头（例如:**-a**),或按语法规则以两个连续的连字符开头（例如:**--version**）。
- 参数（**arguments**）指命令需要执行操作的目标对象。

silde 14

Syntaxe d’une ligne de commande

- Exemples :

```
$ ls -l /home
$ cal 3 2010
```

- Enchaînement de plusieurs commandes
  - séparation des commandes par **;** (point-virgule)
  - exemple :

```
$ ls -l /home ; cal 3 20010
```

命令行语法

- 示例：

```
$ ls -l /home
$ cal 3 2010
```

- 多个命令的串联执行：
  - 用分号（**;**）分隔多个命令。

– 示例：

```
$ ls -l /home ; cal 3 20010
```

Remark 补充命令释义，便于理解上面示例中出现的命令：

- `ls -l /home`：以长格式列出 `/home` 目录下的文件和目录；
- `cal 3 2010`：显示 2010 年 3 月的日历；
- `ls -l /home ; cal 3 20010`：先执行 `ls -l /home`，再执行 `cal 3 20010`（注：`20010` 应为 PPT 笔误，不影响语法逻辑）。

silde 15

Quelques commandes simples

- **date** : affiche la date et l' heure.
- **cal** : affiche le calendrier.
- **who** : affiche la liste des utilisateurs.
- **ls** : donne la liste des fichiers dans le répertoire courant.
- **uname** : affiche le nom et les caractéristiques du système.
- **passwd** : modifie son mot de passe.

一些基础命令

- **date**：显示日期和时间。
- **cal**：显示日历。
- **who**：显示当前登录用户列表。
- **ls**：显示当前目录下的文件列表。
- **uname**：显示系统名称及系统信息。
- **passwd**：修改用户密码。

silde 16

Aide du système : la commande Man

- **Name** : décrit en quelques mots l' objet de la documentation.
- **Synopsis** : présente la syntaxe de la commande.
- **Description** : donne la description précise du fonctionnement de la commande et de chacune de ses options.
- **Files** : fournit la liste des fichiers de configuration ou de données que la commande va mettre en œuvre.
- **See also** : renvoie à des éléments connexes de la documentation.
- **Exit status** : renseigne les codes de retour d' exécution, information utilisée pour la programmation Unix.

系统帮助：man 命令

- **Name**：用几句话描述文档对应的命令用途。

- **Synopsis**：给出命令的语法格式。
- **Description**：详细描述命令的运行机制以及每个选项的作用。
- **Files**：列出该命令所使用的配置文件或数据文件。
- **See also**：指向文档中相关联的其他内容。
- **Exit status**：说明命令执行后的返回值，供 Unix 编程使用。

silde 17

Autres sources de documentation

- La commande **whatis**
  - recherche un mot complet dans la base de données « whatis »
  - exemple :

```
$ whatis date
```

- La commande **apropos**
  - recherche une chaîne de caractères dans la base de données « whatis ». Elle est donc moins restrictive que la commande « whatis »
  - exemple :

```
$ apropos date
```

其他文档来源

- **whatis** 命令：
  - 在 **whatis** 数据库中精确查找完整单词。
  - 示例：

```
$ whatis date
```

- **apropos** 命令：
  - 在 **whatis** 数据库中查找字符串，限制条件比 **whatis** 更宽松。
  - 示例：

```
$ apropos date
```

第四章　Linux 文件系统

在掌握基本命令之后，下一步是理解 Linux 下「一切皆文件」的具体落地方式。文件系统不仅决定了文件如何存储和命名，还影响权限管理、挂载方式以及备份策略。本章将通过多页 `silde` 解释：

- `ext2`、`ext3`、`ext4` 等常见文件系统的基本概念；
- 目录树的全局结构，以及根目录 `/` 下若干关键子目录的职能；
- 绝对路径与相对路径的区别，以及 `cd` 命令在目录树中的「导航」作用。

Remark 建议在阅读本章时，一边对照教材中的树形示意图，一边在真实系统上使用 `ls /`、`ls /home`、`ls /dev` 等命令查看对应目录。可以尝试画出自己实验环境中的简化目录树，并标注出家目录、可执行文件、设备文件所在的位置，从而把抽象结构转化为具体印象。

silde 18

Le système de fichiers

- Un système de fichiers est une façon d'organiser et de stocker une arborescence sur un support (disque, disquette, cd ...) :
  - Chaque OS propriétaire a développé sa propre organisation.
  - On peut faire cohabiter plusieurs systèmes dans des partitions d'un même disque.
  - Linux possède son système appelé **`ext2`** (**`ext3`**, **`ext4`**) mais peut en gérer d'autres. La liste en est donnée dans **`/proc/filesystems`**
- Avec le système Unix, pour l'utilisateur tout est un fichier. Tous les systèmes de fichiers quels que soient leur emplacement physique doivent être intégrés dans l'UNIQUE arborescence logique du système Linux.

文件系统

- 文件系统是在存储介质（硬盘、软盘、光盘等）上组织和存储目录树结构的一种方式：
  - 每个专有操作系统都有自己的文件组织方式。
  - 可以在同一块硬盘的不同分区中并存多个文件系统。
  - Linux 拥有自己的文件系统，如 `ext2`、`ext3`、`ext4`，同时也能管理其他文件系统，支持的列表可在 **`/proc/filesystems`** 中查看。
- 在 Unix 体系中，对用户而言一切皆文件。无论物理位置如何，所有文件系统都必须整合进 Linux 系统唯一的逻辑目录树中。

silde 19

Les types de fichiers

- fichiers ordinaires :

  - texte : courrier, sources des programmes, scripts, configuration ...
  - exécutables : programmes en code binaire
- fichiers répertoires : fichiers conteneurs qui contiennent des références à d'autres fichiers, véritable charpente de l'arborescence, ils permettent d'organiser les fichiers par catégories
- fichiers spéciaux : situés dans **/dev**, ce sont les points d'accès préparés par le système aux périphériques. Le montage va réaliser une correspondance de ces fichiers spéciaux vers leur répertoire "point de montage". par exemple, le fichier **/dev/hda** permet l'accès et le chargement du 1er disque IDE
- fichiers liens symboliques : Ce sont des fichiers qui ne contiennent qu'une référence (un pointeur) à un autre fichier. Cela permet d'utiliser un même fichier sous plusieurs noms sans avoir à le dupliquer sur le disque.

文件类型

- 普通文件：
  - 文本文件：信件、程序源码、脚本、配置文件等；
  - 可执行文件：二进制代码程序。
- 目录文件：作为容器文件，包含对其他文件的引用，是目录树的骨架，用于对文件进行分类管理。
- 特殊文件：位于 **/dev** 目录下，是系统为设备提供的访问入口。挂载操作会将这些特殊文件与它们的挂载点目录建立对应关系。例如：文件 **/dev/hda** 用于访问和加载第一块 IDE 硬盘。
- 符号链接文件：只包含对另一个文件的引用（指针）的文件。可让同一个文件以多个名称使用，而无需在磁盘上重复复制。

silde 20

Quelques répertoires

- **/bin** : les binaires (exécutables) vitaux pour l'utilisateur
- **/dev** : tous les périphériques
- **/home** : les répertoires personnels des utilisateurs
- **/mnt** : les dossiers pour les points de montage temporaires
- **/proc** : accès direct aux paramètres du noyau
- **/root** : répertoire personnel du super utilisateur (root)
- **/var** : les fichiers qui changent fréquemment, tels que : les logs, les mails, les queues d'impression

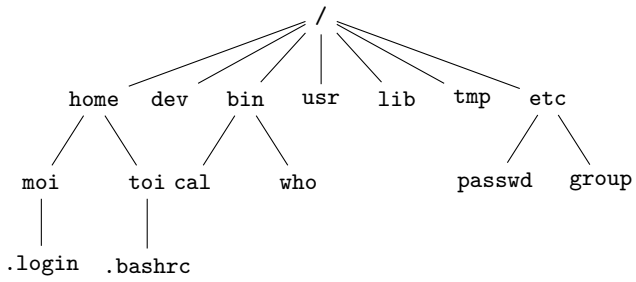
---

常用目录说明

- **/bin**：用户必需的二进制（可执行）文件。
- **/dev**：所有设备文件。
- **/home**：用户个人主目录。
- **/mnt**：临时挂载点目录。
- **/proc**：直接访问内核参数。
- **/root**：超级用户（**root**）的个人主目录。
- **/var**：频繁变动的文件，如日志、邮件、打印队列。

silde 21

Exemple de système de fichiers sous Unix



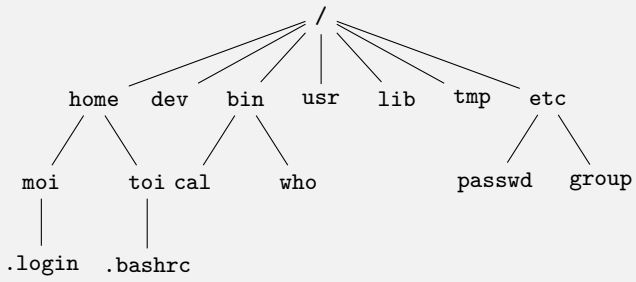
Sous Linux, toute l'arborescence part de « / », appelé « root » ou racine. Dans ce répertoire racine se trouvent tous les dossiers élémentaires

---

Unix 文件系统示例

在 Linux 中，整个目录树从 `/` 开始，称为 `root` 或根目录。根目录下包含所有基础系统目录。

Remark 清晰展示上页 PPT 所示 Unix 文件系统的多叉树结构：



silde 22

Les chemins

- Le chemin d' accès absolu décrit l' itinéraire à emprunter depuis la racine de l' arborescence jusqu' au fichier.
- Le chemin d' accès relatif décrit l' itinéraire à emprunter depuis le répertoire de travail courant. Si le fichier à atteindre est situé au dessous du répertoire de travail, l' utilisateur doit mentionner le chemin qu' il reste à parcourir.
- Le symbole « . » désigne le répertoire courant et le symbole « .. » désigne son père.
- Attention : le symbole « . » devant un nom de fichier signifie qu' il est caché.

路径

- 绝对路径：描述从目录树根目录出发，到达目标文件的完整路径。
- 相对路径：描述从当前工作目录出发，到达目标文件的路径。若目标文件在当前工作目录的下层，用户只需写明后续路径即可。
- 符号 `.` 表示当前目录，符号 `..` 表示父目录（上一级目录）。

- 注意：文件名以 . 开头表示该文件为隐藏文件。

silde 23

Exemple de chemins

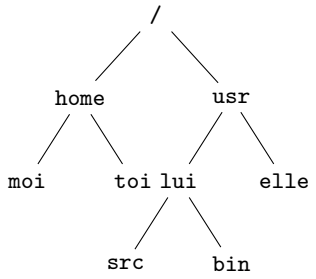
- Par exemple **/usr/bin/cal** est le chemin absolu du fichier **cal**.
- Si le répertoire de travail est le répertoire **/home/moi** :
  - **.login** est équivalent au chemin absolu **/home/moi/.login**
  - **./.login** est équivalent au chemin absolu **/home/moi/.login**
  - **../toi/.cshrc** est équivalent au chemin absolu **/home/toi/.cshrc**
- NB: Le symbole « » désigne le répertoire de connexion de l' utilisateur. Par exemple, si l' utilisateur est **moi**, alors **/.login** correspond au chemin relatif **/home/moi/.login**.

路径示例

- 例如：**/usr/bin/cal** 是文件 **cal** 的绝对路径。
- 若当前工作目录为 **/home/moi**：
  - **.login** 等价于绝对路径 **/home/moi/.login**；
  - **./.login** 等价于绝对路径 **/home/moi/.login**；
  - **../toi/.cshrc** 等价于绝对路径 **/home/toi/.cshrc**。
- 注：符号 表示用户的登录主目录。例如：若用户为 **moi**，则 **/.login** 对应路径 **/home/moi/.login**。

silde 24

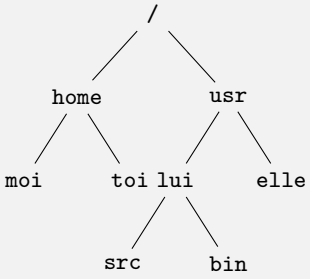
La commande cd



目录树与 cd 命令

- 这张图展示了一个典型的 Linux 目录树，以及使用 **cd**（change directory）命令在目录间跳转的路径。
- 目录树结构大致为：
  - 根目录 **/**
  - **/home** 下有 **moi**、**toi** 两个用户目录；
  - **/usr** 下有 **lui**、**elle** 两个目录，其中 **lui** 下又包含 **src** 与 **bin**。
- 通过不同的绝对路径（如 **/home**、**/home/moi**）和相对路径（如 **..**、**../..**），**cd** 命令可以在这些目录之间灵活跳转。

Remark 目录树结构（对应上页 PPT）：



常见 `cd` 命令跳转示例：

命令	跳转起点	跳转终点	说明
`cd /home`	根目录 `/`	`/home`	使用绝对路径直接进入 `/home` 目录
`cd /home/moi`	根目录 `/`	`/home/moi`	使用绝对路径直接进入 `moi` 目录
`cd /src`	目录 `lui`	目录 `src`	从 `lui` 进入其子目录 `src`（图中简化为 `/src`，实际绝对路径为 `/usr/lui/src`）
`cd ..`	目录 `elle`	目录 `usr`	使用相对路径 `..`（父目录）回到 `/usr` 目录
`cd ../..`	目录 `elle`	根目录 `/`	使用相对路径 `../..`（父目录的父目录）直接回到根目录

核心知识点：

- 绝对路径：以 `/` 开头，从根目录开始描述完整路径，如 `/home/moi`，与当前位置无关；
- 相对路径：以当前目录为起点，如 `..` 表示父目录、`../..` 表示父目录的父目录，结果依赖当前所在目录；
- `cd` 是 Linux 中切换工作目录的核心命令，掌握绝对路径和相对路径的使用是文件系统操作的基础。

silde 25

Les attributs des fichiers

- Commande :

```
$ ls -l fichier
```

- Exemple de sortie :

```
-rwxr-x--- 1 moi etu 300 Jun 02 12:00 fichier
```

- Interprétation des champs :
  - **type** :
    - `-` : fichier ordinaire
    - d : répertoire
    - l : lien symbolique
  - **droits** : permissions d'accès
  - **nombre de liens** : nombre de liens physiques
  - **propriétaire** : nom du propriétaire du fichier
  - **groupe** : nom du groupe propriétaire
  - **taille en octets** : taille du fichier en octets
  - **date et heure de dernière modification** : date et heure de la dernière modification
  - **nom** : nom du fichier

文件属性

- 命令：

```
$ ls -l fichier
```

- 输出示例：

```
-rwxr-x--- 1 moi etu 300 Jun 02 12:00 fichier
```

- 各字段含义：
  - **类型（type）**：
    - -：普通文件
    - d：目录
    - l：符号链接

    权限（droits）：文件访问权限
  - **链接数（nombre de liens）**：硬链接数量
  - **所有者（propriétaire）**：文件的所有者用户名
  - **所属组（groupe）**：文件所属的用户组名
  - **字节大小（taille en octets）**：文件大小（单位：字节）
  - **最后修改时间（date et heure de dernière modification）**：文件最后一次修改的日期和时间
  - **文件名（nom）**：文件名称

silde 26

Quelques commandes pour les fichiers

- **cp** : copie d'un fichier.
- **rm** : détruit un fichier.
- **mv** : change le nom d'un fichier, déplace un fichier.
- **cat**, **more** : affiche le contenu d'un fichier.
- **file** : affiche le type du fichier.
- **cmp**, **comm**, **diff** : compare des fichiers.

文件操作常用命令

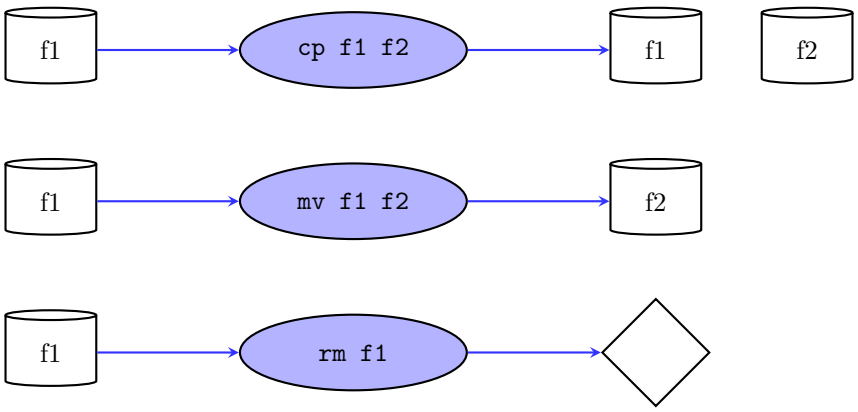
- **cp**：复制文件。
- **rm**：删除文件。
- **mv**：修改文件名、移动文件。
- **cat**、**more**：查看文件内容（**more** 支持分页查看）。
- **file**：查看文件类型（如文本、二进制、脚本等）。
- **cmp**、**comm**、**diff**：对比文件内容（三者对比逻辑略有差异）。

Remark 补充命令使用示例，帮助理解核心文件操作命令的用法：

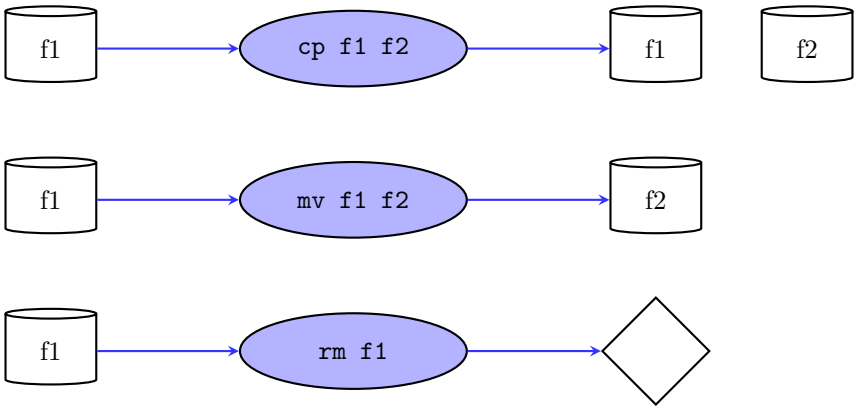
- `cp fichier1 fichier2`：将 fichier1 复制为 fichier2；
- `rm fichier`：删除 fichier 文件（加 `-r` 可删除目录）；
- `mv oldname newname`：将 oldname 重命名为 newname；
- `cat fichier.txt`：直接输出 fichier.txt 的全部内容；
- `more fichier.txt`：分页显示 fichier.txt 的内容（按回车翻行、按空格翻页）；
- `file fichier`：查看 fichier 的具体类型（如 `fichier: ASCII text`）；
- `diff fichier1 fichier2`：对比 fichier1 和 fichier2 的内容差异。

silde 27

Copier, détruire, renommer un fichier



文件的复制、删除与重命名



silde 28

Quelques commandes pour les répertoires

- `pwd` : affiche le répertoire courant.
- `mkdir` : crée un répertoire.
- `rmdir` : supprime un répertoire.

- **du** : affiche la taille d' une arborescence.
- **find** : recherche des fichiers dans une arborescence.

目录操作常用命令

- **pwd**：显示当前工作目录。
- **mkdir**：创建新目录。
- **rmdir**：删除空目录。
- **du**：显示目录及其子目录的磁盘使用空间。
- **find**：在目录树中查找文件。

silde 29

Connaître les droits

- Commande :

```
$ ls -l fichier
```

- Exemple de sortie :

```
-rwxr-x--- 1 moi etu 300 Jun 02 12:00 fichier
```

- Interprétation des droits :
  - **Droits du propriétaire (user)**: `rwx`
  - **Droits du groupe (group)**: `r-x`
  - **Droits pour les autres (others)**: `---`

查看文件权限

- 命令：

```
$ ls -l fichier
```

- 输出示例：

```
-rwxr-x--- 1 moi etu 300 Jun 02 12:00 fichier
```

- 权限解析：
  - **所有者权限 (user)**: `rwx`
  - **所属组权限 (group)**: `r-x`
  - **其他用户权限 (others)**: `---`

silde 30

Droits sur les fichiers et répertoires

- Notation symbolique

- **Qui ?**
  - `u` = user (propriétaire)
  - `g` = group (le groupe)
  - `o` = others (les autres)
- **Permission**
  - `r` = read (consulter le contenu)
  - `w` = write (modifier et détruire)
  - `x` = execute (éxecuter un script ou un programme)

文件与目录权限（符号表示法）

- 符号表示法
  - **作用对象 (Qui)**
    - `u` = user（文件所有者）
    - `g` = group（文件所属组）
    - `o` = others（其他所有用户）
  - **权限类型 (Permission)**
    - `r` = read（读取权限，查看内容）
    - `w` = write（写入权限，修改和删除）
    - `x` = execute（执行权限，运行脚本或程序）

silde 31

Droits sur les fichiers et répertoires

- Notation octale
  - `u`, `g` et `o` sont remplacés par des valeurs octales selon la règle suivante :
    - 0 = aucun droit
    - 1 = exécution (`x`)
    - 2 = écriture (`w`)
    - 4 = lecture (`r`)
  - exemple :

```
$ chmod 400 fichier
```

  - (Cela donne la permission de lecture (4) au propriétaire, et aucun droit aux autres.)

文件与目录权限（八进制表示法）

- 八进制表示法
  - 用八进制数字分别表示 `u`、`g`、`o` 的权限组合，规则如下：
    - 0 = 无任何权限
    - 1 = 执行权限 (`x`)
    - 2 = 写入权限 (`w`)

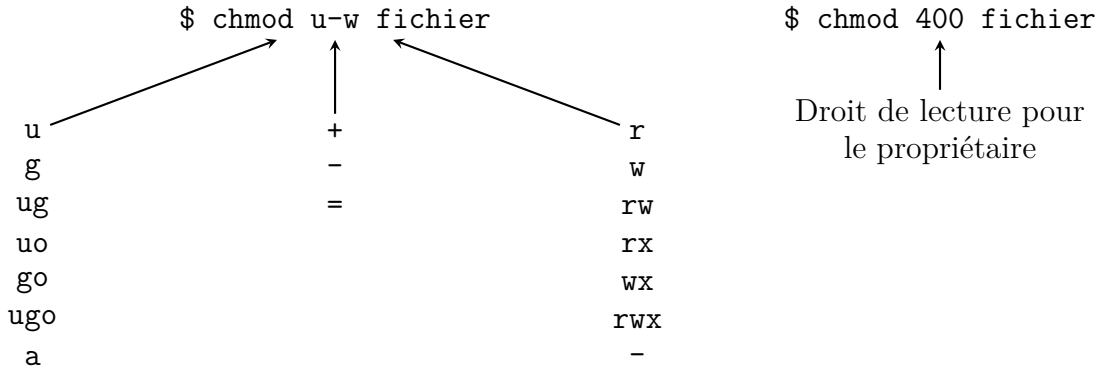
    * 4 = 读取权限 (**r**)
  - 示例：

```
$ chmod 400 fichier
```

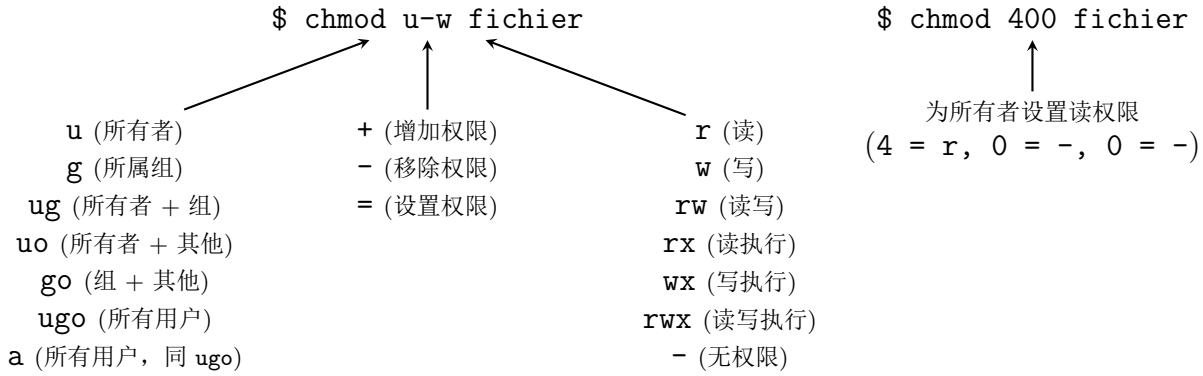
  -（此命令为文件所有者设置了读权限 (4)，其他用户无任何权限。）

silde 32

Droits sur les fichiers et répertoires



文件与目录权限（chmod 命令）



silde 33

Droits sur les fichiers et répertoires : exemples

- `$ chmod 751 exemple.c`

```
-rwxr-x--x 1 retraint users 1829 Mar 18 2002 exemple.c
```

  - `rwx = 4+2+1 = 7` (utilisateur)
  - `r-x = 4+1 = 5` (groupe)
  - `--x = 1` (autres)
- `$ chmod 700 ~`

  Permet de transformer son répertoire de connexion en répertoire privé.

文件与目录权限：示例

- `$ chmod 751 exemple.c`

```
-rwxr-x--x 1 retraint users 1829 Mar 18 2002 exemple.c
```

  - `rwx = 4+2+1 = 7`（所有者权限）
  - `r-x = 4+1 = 5`（所属组权限）
  - `--x = 1`（其他用户权限）

- `$ chmod 700 ~`
  将用户主目录设置为私有目录（仅自己可访问）。

silde 34

Liens physiques et symboliques

- La commande **`ln`** permet de créer un lien pointant vers un fichier (à la manière des raccourcis de Windows).
- Le lien apparaît lorsque l'on passe la commande **`ls -l`**.
- Sans l'option **`-s`**, **`ln`** crée un lien physique (hard link) au lieu d'un lien symbolique.
- Les liens symboliques permettent de ne pas dupliquer l'espace disque utilisé par les fichiers.
- Pour supprimer un lien, on utilise la commande **`rm`**. C'est bien le lien et non le fichier pointé qui est détruit.
- Syntaxe : `$ ln -s fichier_ou_répertoire lien`

硬链接与符号链接

- **`ln`** 命令用于创建指向文件的链接（类似于 Windows 的快捷方式）。
- 使用 **`ls -l`** 命令可以查看到链接信息。
- 若不使用 **`-s`** 选项，**`ln`** 创建的是硬链接（而非符号链接）。
- 符号链接避免了文件在磁盘上的重复拷贝。
- 删除链接使用 **`rm`** 命令，仅删除链接本身，不影响原文件。
- 语法：`$ ln -s 文件或目录链接名`

silde 35

Liens physiques et symboliques : exemple

- `$ ln -s fich fich2`

```
$ ls -l
-rw-r--r-- 1 retraint users 1829 Mar 5 14:25 fich
lrwxrwxrwx 1 retraint users 4 Mar 9 14:03 fich2 -> fich
```

  La dernière ligne indique le lien symbolique entre les fichiers `fich` et `fich2`.

- `$ rm fich`
- `$ cat fich2`

```
cat: fich2: No such file or directory
```

硬链接与符号链接：示例

- $ ln -s fich fich2

```
$ ls -l
-rw-r--r-- 1 retraint users 1829 Mar 5 14:25 fich
lrwxrwxrwx 1 retraint users 4 Mar 9 14:03 fich2 -> fich
```

  最后一行显示了 fich 与 fich2 之间的符号链接。

- $ rm fich
- $ cat fich2

```
cat: fich2: No such file or directory
```

第五章　重定向、管道与错误处理

前一章重点讨论了「静态」的文件和目录，本章则转向「命令运行时的数据流」。在 Shell 环境中，几乎所有命令都遵循「标准输入 → 标准输出 → 标准错误」这一模型：

- 使用 `>`、`>>`、`<`、`<<` 可以将输入输出重定向到文件或从文件读入；
- 使用管道符号 `|` 可以把一条命令的输出实时作为下一条命令的输入；
- 使用 `2>` 这样的写法可以将错误信息单独收集或丢弃（`/dev/null`）。

Remark 建议用一两个具体的小任务来练习本章内容，例如：

- 将 `ls /` 的结果同时保存到文件并分页查看：`ls / > root.txt ; less root.txt`；
- 统计当前目录下包含「.c」的文件个数：`ls | grep "\.c" | wc -l`；
- 对比 `ls -l not-exist` 与 `ls -l not-exist 2>/dev/null` 的终端输出差异，理解「错误重定向」的意义。

silde 36

Les redirections des entrées sorties

Il existe des méthodes pour rediriger les entrées et les sorties des commandes.

- **Les redirections standard**
  - le signe `>` redirige la sortie vers le fichier dont le nom suit (et le remplace)
  - le signe `>>` redirige la sortie vers le fichier dont le nom suit (ajout à la fin du fichier)
  - `<` redirige l'entrée vers le fichier dont le nom suit
  - `<<` permet de donner les arguments à une commande jusqu'au marqueur désigné

输入输出重定向

存在多种方法重定向命令的输入和输出。

- **标准重定向**
  - `>` 将输出重定向到指定文件（覆盖原内容）
  - `>>` 将输出重定向到指定文件（追加到文件末尾）
  - `<` 将输入重定向为指定文件的内容
  - `<<` 将后续输入行作为命令的参数，直到遇到指定的结束标记

silde 37

Les redirections des entrées sorties

- Rediriger l'entrée standard (le clavier)

```
$ commande [options] [arguments] < fichier
```

Les données qui alimentent la commande viennent du fichier.

- Rediriger la sortie standard (l'écran)

```
$ commande [options] [arguments] > fichier
```

Les résultats produits par la commande sont écrits dans le fichier.

输入输出重定向

- 重定向标准输入（键盘）

```
$ 命令 [选项] [参数] < 文件
```

命令从指定文件中读取输入数据。

- 重定向标准输出（屏幕）

```
$ 命令 [选项] [参数] > 文件
```

命令的执行结果写入指定文件。

silde 38

Les redirections : exemples

- `$ cat fich1 > fich2`
- `$ cat <<FIN > sortie`

```
> Bonjour,
> Comment ça va ?
> Au revoir.
> FIN
```

- `$ cat sortie`

```
Bonjour,
Comment ça va ?
Au revoir.
```

重定向示例

- `$ cat fich1 > fich2`
- `$ cat <<FIN > sortie`

```
> 你好,
> 最近怎么样?
> 再见。
> FIN
```

- `$ cat sortie`

```
你好，
最近怎么样？
再见。
```

silde 39

Les redirections des erreurs

- Les commandes envoient les lignes de texte qui ont valeur de résultat sur la sortie standard et les messages qui signalent une erreur de syntaxe ou d'exécution sur la sortie standard des erreurs.
- Pour rediriger la sortie standard des erreurs produites par une commande, il faut mentionner dans la ligne de commande `2>fichier_de_redirection`

```
$ ls -l 2>/dev/null
```

- Pour ignorer les messages d'erreur qui parasitent la lisibilité des résultats, on les redirige vers le périphérique `/dev/null`. Elles sont perdues.

错误重定向

- 命令将正常结果输出到标准输出，将语法错误或执行错误信息输出到标准错误输出。
- 要重定向命令产生的标准错误输出，需在命令行中使用 `2>` 重定向文件

```
$ ls -l 2>/dev/null
```

- 为忽略干扰结果可读性的错误信息，可将其重定向到 `/dev/null` 设备，信息将被丢弃。

silde 40

Les tubes

Le tube, exprimé par le symbole `|`, est un mécanisme de communication qui permet d'échanger des données entre commandes.

`commande1 | commande2 | commande3`

- `$ ls -l | more` : affiche le résultat de la commande `ls`, page par page.
- `$ ls -l | lp` : imprime le résultat de la commande `ls`.

管道

管道由符号 `|` 表示，是一种允许命令之间交换数据的通信机制。

`命令 1 | 命令 2 | 命令 3`

- `$ ls -l | more`：分页显示 `ls` 命令的结果。
- `$ ls -l | lp`：打印 `ls` 命令的结果。

silde 41

Liens physiques et symboliques

- La commande **ln** permet de créer un lien pointant vers un fichier (à la manière des raccourcis de Windows).
- Le lien apparaît lorsque l'on passe la commande **ls -l**.
- Sans l'option **-s**, **ln** crée un lien physique (hard link) au lieu d'un lien symbolique.
- Les liens symboliques permettent de ne pas dupliquer l'espace disque utilisé par les fichiers.
- Pour supprimer un lien, on utilise la commande **rm**. C'est bien le lien et non le fichier pointé qui est détruit.
- `$ ln -s fichier_ou_répertoire lien`

硬链接与符号链接

- **ln** 命令用于创建指向文件的链接（类似于 Windows 的快捷方式）。
- 使用 **ls -l** 命令可以查看到链接信息。
- 若不使用 **-s** 选项，**ln** 创建的是硬链接（而非符号链接）。
- 符号链接避免了文件在磁盘上的重复拷贝。
- 删除链接使用 **rm** 命令，仅删除链接本身，不影响原文件。
- 语法：`$ ln -s 文件或目录链接名`

silde 42

Liens physiques et symboliques : exemple

- `$ ln -s fich fich2`
- `$ ls -l`

```
-rw-r--r-- 1 retraint users 1829 Mar 5 14:25 fich
lrwxrwxrwx 1 retraint users 4 Mar 9 14:03 fich2 -> fich
```

La dernière ligne indique le lien symbolique entre les fichiers `fich` et `fich2`.

- `$ rm fich`
- `$ cat fich2`

```
cat: fich2: No such file or directory
```

硬链接与符号链接：示例

- `$ ln -s fich fich2`
- `$ ls -l`

```
-rw-r--r-- 1 retraint users 1829 Mar 5 14:25 fich
lrwxrwxrwx 1 retraint users 4 Mar 9 14:03 fich2 -> fich
```

最后一行显示了文件 `fich` 和 `fich2` 之间的符号链接。

- `$ rm fich`
- `$ cat fich2`

```
cat: fich2: 没有那个文件或目录
```

第六章　Shell 脚本与参数

在掌握交互式命令行、文件系统与重定向之后，本章开始进入「自动化」阶段：如何把一组命令写入脚本，让系统按照既定逻辑批量执行。Shell 脚本的威力在于：

- 可以把一系列重复的命令固化为脚本文件，方便复用和共享；
- 通过位置参数（`$0`、`$1`、`$*`、`$@`、`#` 等）实现「同一个脚本，处理不同数据」；
- 借助变量、环境变量和只读变量，将一次性实验脚本逐步演化为可维护的工具脚本。

Remark 建议从最简单的脚本开始练习：

- 在家目录中创建 **`hello.sh`**，内容仅为 **`echo "Bonjour, Linux!"`**，依次尝试 **`bash hello.sh`** 与 **`chmod +x hello.sh ; ./hello.sh`**；
- 将脚本扩展为接受参数的版本，例如 **`echo "Bonjour, $1"`**，体会位置参数的作用；
- 通过刻意练习「读配置文件 + 使用环境变量 + 打印诊断信息」，逐步形成良好的脚本编写习惯。

silde 43

Introduction aux scripts

- Le terme « script » désigne un fichier exécutable par le Shell. Il contient des commandes externes du système Unix, des commandes internes et des commentaires.
- Quand on lance l'exécution d'un script, le Shell lit et exécute séquentiellement les lignes du fichier.
- Si le shell détecte une erreur dans une ligne, il arrête la lecture du script et affiche le numéro de ligne de la commande qui a provoqué l'erreur.

脚本入门

- “脚本（script）”指可由 Shell 执行的文件，包含 Unix 外部命令、内部命令和注释。
- 执行脚本时，Shell 按顺序读取并执行文件中的每一行。
- 若 Shell 在某行检测到错误，将停止读取脚本并显示引发错误的命令所在行号。

silde 44

Introduction aux scripts

- L'environnement d'un utilisateur (type de terminal, choix du prompt, définition des touches d'effacement des caractères, etc.) est construit à partir de deux scripts :
  - le script `/etc/profile` commun à tous les utilisateurs.
  - le script `~/.bash_profile` qui se trouve dans le répertoire de connexion de l'utilisateur.

脚本入门

- 用户环境（终端类型、提示符选择、字符删除键定义等）由两个脚本构建：
  - `/etc/profile`：所有用户共享的全局配置文件。
  - `~/.bash_profile`：位于用户主目录下的个人配置文件。

silde 45

Les commentaires

- Dans une ligne de commande, le caractère **#** indique le début d'un commentaire qui se poursuit jusqu'à la fin de la ligne.
- Le pseudo commentaire **#!**, placé en début de script, permet de désigner explicitement le shell qui doit exécuter le script.
- Exemple :

```
#!/bin/bash # ce script est exécuté par bash
# Ce script affiche la liste des processus
ps
```

注释

- 在命令行中，字符 **#** 表示注释开始，直到行末。
- 特殊注释 **#!** 放在脚本开头，显式指定执行该脚本的 Shell。
- 示例：

```
#!/bin/bash # 此脚本由 bash 执行
# 显示进程列表
ps
```

silde 46

Exécution d'un script

- Exécution d'un script :

```
$ bash lescript
$ bash <lescript
$ chmod +x ; ./lescript
```

- Activation/désactivation du mode trace dans le script : il faut inclure la commande **set -x** dans le script.
- Exécution d'un script avec l'affichage de la trace des commandes :

```
$ bash -xv lescript
```

脚本的执行

- 执行脚本的几种方式：

```
$ bash 脚本名
$ bash <脚本名
$ chmod +x 脚本名 ; ./脚本名
```

- 在脚本中启用/禁用跟踪模式：在脚本内加入命令 **`set -x`**。
- 执行脚本并显示命令跟踪信息：

```
$ bash -xv 脚本名
```

silde 47

Les paramètres

- Dans la terminologie du Shell, un paramètre désigne toute entité pouvant contenir une valeur. Le Shell distingue trois classes de paramètres :
  - les variables, identifiées par un nom (ex: `a`, `PATH`)
  - les paramètres de position, identifiés par un numéro (ex: `0`, `1`, `12`)
  - les paramètres spéciaux, identifiés par un caractère spécial (ex: `#`, `?`, `$`)
- Le mode d'affectation d'une valeur est spécifique à chaque classe de paramètres. Pour obtenir la valeur d'un paramètre, on placera toujours le caractère **`$`** devant sa référence, et cela quelle que soit sa classe.
- Exemple : `$ echo $PATH` affiche la valeur de la variable `PATH`.

参数

- 在 Shell 术语中，参数指任何可包含值的实体。Shell 区分三类参数：
  - 变量：由名称标识（如 `a`、`PATH`）
  - 位置参数：由数字标识（如 `0`、`1`、`12`）
  - 特殊参数：由特殊字符标识（如 `#`、`?`、`$`）
- 每类参数的赋值方式不同。获取参数值时，无论哪类，均需在引用前加 **`$`**。
- 示例：`$ echo $PATH` 显示变量 `PATH` 的值。

silde 48

Les variables

- Une variable est identifiée par un nom, c'est-à-dire une suite de lettres, de chiffres ou de caractères ne commençant pas par un chiffre. Les lettres majuscules et minuscules sont différenciées.
- Les variables peuvent être classées en trois groupes :
  - les variables utilisateur (ex: `a`, `valeur`)
  - les variables prédéfinies du Shell (ex: `PS1`, `PATH`, `REPLY`, `IFS`)
  - les variables prédéfinies de commandes Unix (ex: `TERM`)
- En général, les noms des variables utilisateur sont en lettres minuscules tandis que les noms des variables prédéfinies (du Shell ou de commandes Unix) sont en majuscules.

变量

- 变量由名称标识，名称是由字母、数字或下划线组成的字符串，且不能以数字开头。区分大小写。
- 变量可分为三组：
  - 用户变量（如 a、valeur）
  - Shell 预定义变量（如 PS1、PATH、REPLY、IFS）
  - Unix 命令预定义变量（如 TERM）
- 通常，用户变量名使用小写字母，而预定义变量名（Shell 或 Unix 命令）使用大写字母。

silde 49

Les variables

- L’utilisateur peut affecter une valeur à une variable en utilisant :
  - l’opérateur d’affectation =
  - la commande interne read
- Affectation directe :

```
variable=valeur
```

- Valeur d’une variable :

```
$variable
```

- Exemple :

```
$ x=bon
$ echo $x
bon
$ y=jour
$ x=$x$y
$ echo $x
bonjour
```

变量

- 用户可以通过以下方式为变量赋值：
  - 赋值运算符 =
  - 内部命令 read
- 直接赋值：

```
变量名=值
```

- 引用变量值：

```
$变量名
```

- 示例：

```
$ x=bon
$ echo $x
bon
$ y=jour
$ x=$x$y
$ echo $x
bonjour
```

silde 50

Les variables

- Affectation par lecture :

```
read [ var1 var2 ... ]
```

Exemple :

```
$ read x y
bon jour
$ echo $x$y
bonjour
```

- Lorsque la commande interne `read` est utilisée sans argument, la ligne lue est enregistrée dans la variable prédéfinie du shell `REPLY`. Exemple :

```
$ read
bonjour tout le monde
$ echo $REPLY
bonjour tout le monde
```

变量

- 通过读取赋值：

```
read [ 变量1 变量2 ... ]
```

示例：

```
$ read x y
bon jour
$ echo $x$y
bonjour
```

- 当 `read` 不带参数使用时，读入的行将保存在 Shell 预定义变量 `REPLY` 中。示例：

```
$ read
bonjour tout le monde
$ echo $REPLY
bonjour tout le monde
```

silde 51

Les variables

- L'option `-p` de `read` affiche une chaîne d'appel avant d'effectuer la lecture ; la syntaxe à utiliser est :

```
read -p chaîne_d_appel [ var ... ]
```

Exemple :

```
$ read -p "Entrez votre prenom : " prenom
Entrez votre prenom : Eric
$ echo $prenom
Eric
```

- Liste des variables :
  La commande `set` liste l'ensemble des variables.
- Suppression d'une variable :

```
unset variable
```

变量

- `read` 命令的 `-p` 选项在执行读取前显示提示字符串；语法为：

```
read -p 提示字符串 [ 变量 ... ]
```

示例：

```
$ read -p "请输入您的名字: " prenom
请输入您的名字: Eric
$ echo $prenom
Eric
```

- 变量列表：
  命令 `set` 可列出所有变量。
- 删除变量：

```
unset 变量名
```

silde 52

Définition de séparateur de champs : IFS

- La commande `read` lit une ligne entière et utilise la variable `IFS` pour décomposer la ligne en mots qu'elle va affecter aux différentes variables de lecture qui lui sont passées en argument.
- Par défaut, la variable `IFS` est initialisée avec les caractères <Espace>, <tabulation> et <Saut de ligne>.
- Modification de la variable `IFS` :

```
IFS=:
```

字段分隔符：IFS

- read 命令读取整行，并利用变量 IFS 将该行分解为单词，然后依次赋值给作为参数给出的各个变量。
- 默认情况下，IFS 包含空格、制表符和换行符。
- 修改变量 IFS 示例：

```
IFS=:
```

silde 53

Les variables en lecture seule

- Pour définir une variable dont la valeur ne doit pas être modifiée (appelée constante dans d'autres langages de programmation), on utilise la syntaxe :

```
declare -r nom=valeur [ nom=valeur ... ]
```

- Exemple : `$ declare -r mess=bonjour`
- Une tentative de modification de la valeur d'une constante provoque une erreur.

```
$ mess=salut
-bash: mess: readonly variable
```

- Pour connaître la liste des constantes définies : `declare -r`

只读变量

- 定义值不可修改的变量（其他编程语言中的常量），使用语法：

```
declare -r 变量名=值 [ 变量名=值 ... ]
```

- 示例：`$ declare -r mess=bonjour`
- 试图修改常量值将引发错误：

```
$ mess=salut
-bash: mess: 只读变量
```

- 查看已定义的常量列表：`declare -r`

silde 54

Les variables d'environnement

- Exemple :

```
$ var_locale=alpha
$ var_environ=gamma
$ export var_environ
```

- La commande `export` permet à un shell de placer une variable dans l'environnement. Syntaxe :

```
$ export variable[=valeur]
```

- La commande `env` permet de lister l'ensemble des variables exportées.

环境变量

- 示例：

```
$ var_locale=alpha
$ var_environ=gamma
$ export var_environ
```

- `export` 命令使 Shell 将变量放入环境中。语法：

```
$ export 变量名[=值]
```

- `env` 命令列出所有已导出的变量。

silde 55

Les paramètres

- Un script est à l'image d'une commande. Il est possible de fournir, sur la ligne de commandes, les arguments nécessaires à son exécution.
- Les arguments sont affectés aux variables `0`, `1`, ⋯, `9`, `10`, `11`, ⋯
- La variable `*` désigne l'ensemble des paramètres sous la forme d'un seul argument.
- La variable `@` désigne l'ensemble des paramètres, un argument par paramètre.

参数

- 脚本类似于命令，可以在命令行中提供执行所需的参数。
- 参数被依次赋给变量 `0`、`1`、⋯、`9`、`10`、`11`⋯⋯
- 变量 `*` 表示所有参数合并为一个字符串。
- 变量 `@` 表示所有参数，每个参数作为独立的个体。

silde 56

Les paramètres

- La variable `#` désigne le nombre de paramètres passés au script.
- La commande `shift` permet de décaler les paramètres : la valeur de la variable `1` est remplacée par celle de `2`, celle de `2` par celle de `3`, etc.

```
shift [n] # décale de n paramètres
```

- La commande interne `set` permet de remplacer tous les paramètres.

```
$ set un deux trois ; echo $*
un deux trois
```

参数

- 变量 # 表示传递给脚本的参数个数。
- shift 命令用于偏移参数：变量 1 的值被 2 替代，2 被 3 替代，以此类推。

```
shift [n] # 偏移 n 个参数
```

- 内部命令 set 可以重新设置所有参数。

```
$ set un deux trois ; echo $*
un deux trois
```

silde 57

Suppression des ambigüités

- Pour éviter les ambiguïtés dans l' interprétation des références de paramètres, on utilise la syntaxe ${paramètre}.
- Exemples :

```
$ set un deux trois quatre cinq six sept huit neuf dix onze douze
$ echo $11
un1
$ echo ${11} # pour obtenir la valeur du onzième paramètre de position
onze
```

消除歧义

- 为避免参数引用时的歧义，应使用 ${参数} 语法。
- 示例：

```
$ set un deux trois quatre cinq six sept huit neuf dix onze douze
$ echo $11
un1
$ echo ${11} # 获取第十一个位置参数的值
onze
```

silde 58

Substitution de commandes

- Une commande cmd entourée par une paire de parenthèses () précédées d'un caractère $ est exécutée par le

shell puis la chaîne `$(cmd)` est remplacée par les résultats de la commande `cmd` écrits sur la sortie standard. Ces résultats peuvent alors être affectés à une variable ou bien servir à initialiser des paramètres de position.

- Exemple :

```
$ pwd
/home/retraint
$ repert=$(pwd)
$ echo mon repertoire est $repert
mon repertoire est /home/retraint
```

命令替换

- 用 `$()` 括起来的命令会被 Shell 执行，并将该命令输出到标准输出的结果替换到原位置。这些结果可以赋值给变量，或用于初始化位置参数。
- 示例：

```
$ pwd
/home/retraint
$ repert=$(pwd)
$ echo 我的目录是 $repert
我的目录是 /home/retraint
```

silde 59

Substitution de commandes

- Lorsque la substitution de commande est utilisée avec la commande interne `set`, l' erreur suivante peut se produire :

```
$ set $(ls -l .bashrc)
bash: set: -w: invalid option
```

`$ ls -l .bashrc` affiche `-rw-r--r-- 1 retraint users 1342 nov 30 2009 .bashrc`. Le premier mot issu de la substitution commence par un caractère tiret : la commande interne `set` l'interprète comme une option, ce qui provoque une erreur. Pour résoudre ce type de problème, on double le caractère tiret. Exemple :

```
$ set -- $(ls -l .bashrc)
$ echo $1
-rw-r--r--
```

命令替换

- 当命令替换与内部命令 `set` 一起使用时，可能发生如下错误：

```
$ set $(ls -l .bashrc)
bash: set: -w: 无效选项
```

`$ ls -l .bashrc` 输出 `-rw-r--r-- 1 retraint users 1342 nov 30 2009 .bashrc`。替换结果的第一个单词以连字符开头，`set` 将其解释为选项，导致错误。解决方法是在连字符前加双横线：

```
$ set -- $(ls -l .bashrc)
$ echo $1
-rw-r--r--
```

silde 60

Substitution de commandes

- Les substitutions de commandes peuvent être imbriquées. Exemple :

```
$ set $(ls $(pwd)/retraint)
$ echo $# # nombre d'entrées du répertoire retraint
3
```

- La substitution de commande permet également de capter les résultats écrits par un fichier shell. Exemple :

```
$ lescript
38
$ resultat=$(lescript)
```

La variable `resultat` contient la chaîne `38`.

命令替换

- 命令替换可以嵌套使用。示例：

```
$ set $(ls $(pwd)/retraint)
$ echo $# # retraint 目录中的条目数
3
```

- 命令替换也可捕获 Shell 脚本的输出。示例：

```
$ lescript
38
$ resultat=$(lescript)
```

变量 `resultat` 包含字符串 `38`。

silde 61

Le code retour

- Une commande Unix ou un script qui se termine émettent un code de retour numérique.
- La variable prédéfinie `?` est automatiquement initialisée avec le code retour de la dernière commande exécutée.

```
$ who | grep thomas
$ echo $? # thomas est-il connecté ?
```

```
0 # thomas est connecté
```

返回码

- 每个 Unix 命令或脚本执行结束后都会返回一个数值作为返回码。
- 预定义变量 ? 自动保存最后一条命令的返回码。

```
$ who | grep thomas
$ echo $? # thomas 是否已登录?
0 # thomas 已登录
```

第七章　流程控制与条件测试

前几章主要侧重命令和脚本的「成分表」（变量、参数、重定向等），本章则聚焦于「控制流程」：

- 使用 `if`、`case` 根据条件分支执行不同命令序列；
- 使用 `for`、`while`、`until` 等循环结构实现重复性任务；
- 通过 `test` 命令和逻辑运算符，对文件属性、字符串和数值进行条件判断。

Remark 在 Shell 脚本中，「是否成功执行」通常通过返回码（`$?` 为 0 或非 0）来表达，而不是通过布尔类型。建议在练习本章内容时，配合 `echo $?` 与 `set -x`（跟踪执行过程）一起使用，观察不同条件表达式在各种输入下的返回码变化。也可以尝试改写课件中的示例脚本，在关键分支处打印提示信息，以便更直观地理解程序的执行路径。

silde 62

Les instructions de contrôle

- Comme tous les langages de programmation, le langage shell possède des instructions de contrôle telles que par exemples :
  - L'alternative : l'instruction `if`.
  - Les structures itératives : les instructions `for`, `while`, `until`.
  - Le choix multiple : l'instruction `case`.

控制指令

- 与所有编程语言一样，Shell 语言也提供控制指令，例如：
  - 分支：`if` 语句。
  - 循环结构：`for`、`while`、`until` 语句。
  - 多路选择：`case` 语句。

silde 63

if : l'alternative

- Syntaxe :

```
if condition
then
    bloc de commandes
elif condition
then
```

```
    bloc de commandes
else
    bloc de commandes
fi
```

---

if 条件分支

- 语法：

```
if 条件
then
    命令块
elif 条件
then
    命令块
else
    命令块
fi
```

silde 64

L’alternative : exemple

- Exemple :

```
if test -f $1
then
    file $1
else
    echo "le fichier $1 n'existe pas"
fi
```

Si le fichier dont le nom est passé en paramètre existe, alors il affiche le type de son contenu.

- Il est possible de positionner les mots clés `if` et `then` sur la même ligne en les séparant par un point virgule :

```
if test -f $1 ; then file $1 ; fi
```

分支示例

- 示例：

```
if test -f $1
then
    file $1
else
    echo "文件 $1 不存在"
fi
```

如果通过参数传入的文件名存在，则显示其文件类型。

- 可以将 `if` 和 `then` 放在同一行，用分号分隔：

```
if test -f $1 ; then file $1 ; fi
```

silde 65

L' alternative && et ||

- Les opérateurs `&&` et `||` permettent de conditionner l' exécution d' une commande par le code retour de celle qui précède.
- `commande1 && commande2`
  - équivaut à : `if commande1 ; then commande2 ; fi`
  - `commande2` est exécutée si `commande1` a renvoyé 0.
- `commande1 || commande2`
  - Équivaut à : `if commande1 ; then ; else commande2 ; fi`
  - `commande2` est exécutée si `commande1` a renvoyé 1.

逻辑与 && 和逻辑或 ||

- 运算符 `&&` 和 `||` 根据前一条命令的返回码决定是否执行后一条命令。
- `命令 1 && 命令 2`
  - 等价于：`if 命令 1 ; then 命令 2 ; fi`
  - 当 `命令 1` 返回 0（成功）时执行 `命令 2`。
- `命令 1 || 命令 2`
  - 等价于：`if 命令 1 ; then ; else 命令 2 ; fi`
  - 当 `命令 1` 返回非 0（失败）时执行 `命令 2`。

silde 66

La commande test

- La syntaxe
  - `if test expression ; then ... fi`
  - `if [ expression ] ; then ... fi`
- La commande `test` n'affiche pas de résultat sur la sortie standard, elle renvoie le code retour 0 si l'expression est vraie et 1 si elle est fausse.

---

test 命令

- 语法：
  - `if test 表达式; then ... fi`

  - `if [ 表达式] ; then ... fi`
- `test` 命令不在标准输出显示结果，仅返回状态码：表达式为真时返回 0，为假时返回 1。

silde 67

Test d' un attribut de fichier

- `-f fichier` : vrai si le fichier existe et qu' il est ordinaire.
- `-e fichier` : vrai si le fichier existe.
- `-r fichier` : vrai si le fichier existe et qu' il est accessible en lecture.
- `-w fichier` : vrai si le fichier existe et qu' il est accessible en écriture.

文件属性测试

- `-f` 文件：若文件存在且为普通文件，则为真。
- `-e` 文件：若文件存在，则为真。
- `-r` 文件：若文件存在且可读，则为真。
- `-w` 文件：若文件存在且可写，则为真。

silde 68

Test sur des chaînes de caractères

- `-z chaîne` : vrai si la longueur de la chaîne est 0.
- `-n chaîne` : vrai si la longueur de la chaîne est différente de 0.
- `chaîne1 = chaîne2` : vrai si `chaîne1` est identique à `chaîne2`.
- `chaîne1 != chaîne2` : vrai si `chaîne1` est différente de `chaîne2`.

字符串测试

- `-z` 字符串：若字符串长度为 0，则为真。
- `-n` 字符串：若字符串长度不为 0，则为真。
- 字符串 `1 =` 字符串 `2`：若两个字符串相等，则为真。
- 字符串 `1 !=` 字符串 `2`：若两个字符串不等，则为真。

silde 69

Test sur les nombres

- `arg1 -eq arg2` : vrai si `arg1` est égal à `arg2`.
- `arg1 -ne arg2` : vrai si `arg1` est différent de `arg2`.
- `arg1 -lt arg2` : vrai si `arg1` est strictement inférieur à `arg2`.
- `arg1 -gt arg2` : vrai si `arg1` est strictement supérieur à `arg2`.
- `-le` et `-ge` pour inférieur ou égal et supérieur ou égal.

数值测试

- `arg1 -eq arg2`：若 `arg1` 等于 `arg2`，则为真。
- `arg1 -ne arg2`：若 `arg1` 不等于 `arg2`，则为真。
- `arg1 -lt arg2`：若 `arg1` 小于 `arg2`，则为真。
- `arg1 -gt arg2`：若 `arg1` 大于 `arg2`，则为真。
- `-le` 表示小于等于，`-ge` 表示大于等于。

silde 70

Opérateurs logiques

- `!expression_logique` : vrai si `expression_logique` est fausse.
- `expression_logique1 -a expression_logique2` : vrai si les deux expressions sont vraies.
- `expression_logique1 -o expression_logique2` : vrai si l'une des deux expressions est vraie.

逻辑运算符

- `! 逻辑表达式`：若逻辑表达式为假，则为真（逻辑非）。
- `表达式 1 -a 表达式 2`：若两个表达式均为真，则为真（逻辑与）。
- `表达式 1 -o 表达式 2`：若其中一个表达式为真，则为真（逻辑或）。

silde 71

La structure case

- Syntaxe :

```
case paramètre in
    choix1.1 | choix1.2 )
        commande1; commande2;;
    choix2 )
        commandes;;
    * )
        commandes;;
esac
```

- Chaque valeur du paramètre est terminée par `)`. Si plusieurs valeurs sont à traiter de la même manière, on les sépare par le signe `|`.
- Chaque bloc de commandes se termine par `;;`. Le choix par défaut se marque par `* )` et est toujours situé en dernier.

case 多路分支

- 语法：

```
case 参数 in
    模式1.1 | 模式1.2 )
        命令1; 命令2;;
    模式2 )
        命令块;;
    * )
        命令块;;
esac
```

- 每个模式值以) 结尾。若多个模式执行相同操作，用 | 分隔。
- 每个命令块以 ;; 结尾。默认分支使用 *)，且必须放在最后。

silde 72

La structure case : exemple

- Exemple (script supprime) :

```
$ cat supprime
echo "voulez vous supprimer le fichier ?"
read reponse
case $reponse in
    O | o )
        rm $1;;
    N | n )
        echo "le fichier n'est pas supprimé";;
    * )
        echo "réponse incorrecte";;
esac
```

---

case 示例

- 示例（脚本 supprime）:

```
$ cat supprime
echo "是否要删除文件？"
read reponse
case $reponse in
    O | o )
        rm $1;;
    N | n )
        echo "文件未被删除";;
    * )
        echo "无效回答";;
esac
```

silde 73

La boucle tant que

- Syntaxe :

```
while condition
do
    commandes
done
```

- Exemple :

```
while [ -r $1 ]
do
    cat $1 >> result
    shift
done
```

Concatène dans le fichier `result` l' ensemble des fichiers dont les noms sont donnés en argument.

---

while 循环

- 语法:

```
while 条件
do
    命令块
done
```

- 示例:

```
while [ -r $1 ]
do
    cat $1 >> result
    shift
done
```

将所有作为参数传入的可读文件内容追加到 `result` 文件中。

silde 74

La boucle répéter jusqu'à

- Syntaxe :

```
until condition
do
    commandes
done
```

- Exemple :

```
until [ ! -r $1 ]
do
    cat $1 >> concat
    shift
done
```

Notez la négation du test grâce à l'opérateur de négation.

---

until 循环

- 语法：

```
until 条件
do
    命令块
done
```

- 示例：

```
until [ ! -r $1 ]
do
    cat $1 >> concat
    shift
done
```

注意测试条件中使用了否定运算符 `!`。

silde 75

La boucle for

- Syntaxe :

```
for paramètre [ in liste ]
do
    commandes
done
```

- Exemple :

```
for i in `ls` # ou $(ls)
do
    cp $i /tmp/$i
    echo "$i copié"
done
```

`` `ls` `` signifie le résultat de l'exécution de `ls`. Recopie chaque fichier du répertoire courant dans `/tmp`.

---

for 循环

- 语法：

```
for 变量 [ in 列表 ]
do
    命令块
done
```

- 示例：

```
for i in `ls` # 或 $(ls)
do
    cp $i /tmp/$i
    echo "$i 已复制"
done
```

`` `ls` `` 表示执行 `ls` 命令的结果。将当前目录下的每个文件复制到 `/tmp` 目录。

silde 76

Les sauts inconditionnels

- Exemple d'utilisation de `continue` et `break` dans une boucle :

```
while ... ; do
    if ... ; then
        continue # passe à l'itération suivante
    fi
    if ... ; then
        break # sort de la boucle
    fi
done
...
```

无条件跳转

- 在循环中使用 `continue` 和 `break` 的示例：

```
while ... ; do
    if ... ; then
        continue # 跳过本次循环剩余部分，进入下一次迭代
    fi
    if ... ; then
        break # 跳出整个循环
    fi
done
...
```

第八章　算术扩展、函数与数组

当脚本规模逐渐增大时，仅靠顺序执行与简单变量已经难以组织复杂逻辑。本章介绍三个重要能力：

- 利用 `declare -i`、`(( ))`、`$(( ))` 和 C 风格 `for` 循环进行整数运算与计数；
- 使用 Shell 函数对脚本逻辑分块，实现「定义一次、多处调用」，并通过 `local` 管理局部变量；
- 借助数组保存成组数据，配合循环完成批量处理任务。

Remark 建议在阅读本章时，选取一个实际小任务作为「项目练习」，例如：

- 编写一个脚本统计若干日志文件中错误行的数量，使用循环与算术扩展累计计数；
- 将「统计某个目录下文件个数并输出摘要」封装为函数，尝试在同一脚本中多次调用；
- 使用数组保存一组重要目录路径，通过 `for` 循环依次检查其磁盘使用情况。

通过把算术、函数与数组结合在一个小项目中使用，可以显著提升对这些抽象概念的掌握程度。

silde 77

L' arithmétique

- Pour définir et initialiser une ou plusieurs variables de type entier, on utilise la syntaxe suivante :

```
declare -i nom[=expr_arith] [ nom[=expr_arith] ... ]
```

- Exemples :

```
$ declare -i a=1
$ declare -i a b
$ read a
```

- Il n'est pas nécessaire de définir une variable avant de l'utiliser.

算术运算

- 定义并初始化一个或多个整型变量，使用如下语法：

```
declare -i 变量名[=算术表达式] [ 变量名[=算术表达式] ... ]
```

- 示例：

```
$ declare -i a=1
$ declare -i a b
$ read a
```

- 变量在使用前不必事先声明。

silde 78

La commande interne (( ))

- Cette commande interne est également utilisée pour effectuer des opérations arithmétiques.
- Syntaxe : `(( expr_arith ))`
- Exemples :

```
$ set $(date)
$ declare -i a=0
$ ((a=$2+1))
$ echo $a
11
```

内部命令 (( ))

- 内部命令 `(( ))` 也用于执行算术运算。
- 语法：`(( 算术表达式))`
- 示例：

```
$ set $(date)
$ declare -i a=0
$ ((a=$2+1))
$ echo $a
11
```

silde 79

La commande interne (( ))

- Pour obtenir la valeur de l'expression arithmétique, on utilise la syntaxe : `$(( expr_arith ))`
- Exemple : `echo $(( 7 * 3 ))`
- Substitutions de commandes et substitutions de paramètres peuvent être présentes dans une commande interne `(( ))` ou bien dans une substitution d' expression arithmétique `$(( ))` si le résultat aboutit à un nombre entier.
- Par exemple, dans l' expression `$(( $(ls -l | wc -l) -1 ))` le résultat du pipeline `ls -l | wc -l` est interprété comme un nombre.

内部命令 (( ))

- 要获取算术表达式的值，可使用语法：`$(( 算术表达式))`
- 示例：`echo $(( 7 * 3 ))`
- 命令替换和参数替换可以出现在 `(( ))` 或 `$(( ))` 中，只要结果能解释为整数。
- 例如，`$(( $(ls -l | wc -l) -1 ))` 中管道 `ls -l | wc -l` 的结果被当作数值处理。

silde 80

Les opérateurs

- On retrouve l' ensemble des opérateurs du langage C :
  - `a++` et `a--` : post-incrémentation, post-décrémentation
  - `++a` et `--a` : pré-incrémentation, pré-décrémentation
  - `-a` et `+a` : moins unaire, plus unaire
  - `a**b` : exponentiation a^b
  - `a*b`, `a/b` et `a%b` : multiplication, division entière, reste

---

运算符

- 支持 C 语言中的所有运算符：
  - `a++` 和 `a--`：后置自增、自减
  - `++a` 和 `--a`：前置自增、自减
  - `-a` 和 `+a`：一元负号、正号
  - `a**b`：乘方 a^b
  - `a*b`、`a/b` 和 `a%b`：乘法、整数除法、取余

silde 81

Structure for pour les expressions arithmétiques

- Bash a introduit une nouvelle structure `for` adaptée aux traitements des expressions arithmétiques, itération issue du langage C.
- Syntaxe :

```
for (( expr_arith1 ; expr_arith2 ; expr_arith3 ))
do
    suite_cmd
done
```

- Exemple :

```
declare -i x y
for (( x=1, y=10 ; x<4 ; x++, y-- ))
do
    echo $(( x*y ))
done
```

算术表达式的 for 循环

- Bash 引入了类似于 C 语言的 `for` 循环结构，用于处理算术表达式。
- 语法：

```
for (( 算术表达式1 ; 算术表达式2 ; 算术表达式3 ))
do
    命令块
done
```

- 示例：

```
declare -i x y
for (( x=1, y=10 ; x<4 ; x++, y-- ))
do
    echo $(( x*y ))
done
```

silde 82

Les fonctions

- Syntaxe :

```
function Nom {
    lignes de code ;
}
```

ou

```
Nom ( ) {
    lignes de code ;
}
```

- Déclaration des variables locales
  - `local -i var` : déclaration d'un entier `var`
  - `local -a tab` : définition d'un tableau

函数

- 语法：

```
function 函数名 {
    代码行 ;
}
```

或

```
函数名 ( ) {
    代码行 ;
}
```

- 局部变量声明：
  - `local -i` 变量名：声明整型局部变量
  - `local -a` 数组名：定义数组

silde 83

Les fonctions

- Une fonction est exécutée en invoquant son nom suivi d'éventuels paramètres qui seront référencés de la même manière qu'un script (`$0`, `$1`, ⋯).
- La commande `unset -f` permet de supprimer une fonction.
- La commande `return` met fin à une fonction. Le code retour est stocké, comme le retour d'un script, dans la variable `?` du shell.

函数

- 调用函数时直接使用函数名，后可跟参数，参数引用方式与脚本相同（`$0`、`$1`⋯⋯）。
- `unset -f` 命令用于删除函数。
- `return` 命令结束函数执行，其返回码存入 Shell 的 `?` 变量中（类似脚本的返回码）。

silde 84

Exporter une fonction

- Pour qu'une fonction puisse être exécutée par un programme Shell différent de celui où elle a été définie, il est nécessaire d'exporter cette fonction. On utilise la commande interne `export`.

```
export -f nomfct ...
```

- Pour que l'export fonctionne, le sous-shell qui exécute la fonction doit avoir une relation de descendance avec le programme Shell qui exporte la fonction.

导出函数

- 若希望函数能被定义它的 Shell 之外的另一个 Shell 程序执行，需要导出该函数。使用内部命令 `export:`

```
export -f 函数名 ...
```

- 导出成功的前提是执行函数的子 Shell 必须是导出函数的 Shell 的后代进程。

silde 85

Les tableaux

- Pour créer un tableau, on utilise généralement l'option `-a` (comme array) de la commande `declare` :

```
declare -a nomtab ...
```

  Le tableau `nomtab` est simplement créé mais ne contient aucune valeur : le tableau est défini mais n'est pas initialisé.
- Pour connaître les tableaux définis : `declare -a`
- Pour définir et initialiser un tableau :

```
declare -a nomtab=( val0 val1 ... )
```

- Comme en langage C, l'indice du tableau débute toujours à 0.

数组

- 创建数组通常使用 `declare` 命令的 `-a` 选项：

```
declare -a 数组名 ...
```

此时数组 数组名被创建，但不包含任何值：数组已定义但未初始化。

- 查看已定义的数组：`declare -a`
- 定义并初始化数组：

```
declare -a 数组名=( 值0 值1 ... )
```

- 与 C 语言一样，数组下标从 0 开始。

silde 86

Les tableaux

- Exemple :

```
$ tableau[0]=un
$ tableau[1]=deux
$ tableau[2]=trois
$ echo ${tableau[0]}
un
$ tableau[1]=six
$ echo ${tableau[*]}
un six trois
$ echo ${#tableau[*]}
3
```

数组示例

- 示例：

```
$ tableau[0]=un
$ tableau[1]=deux
$ tableau[2]=trois
$ echo ${tableau[0]}
un
$ tableau[1]=six
$ echo ${tableau[*]}
un six trois
$ echo ${#tableau[*]}
3
```